# GAME PROJECT — PHÂN RÃ CÔNG VIỆC VÀ KIẾN TRÚC HỆ THỐNG

## Mục lục


**1 Tổng quan** **2**

**2 Ranh giới trách nhiệm** **2**

**3 CORE SIMULATION** **3**

3.1 Mục tiêu . . . . . 3

**4 CORE — ECS FOUNDATION** **4**

**5 CORE — DATA STORAGE** **4**

**6 CORE — CHUNK CONFIGURATION** **5**

**7 CORE — ENTITY / OBJECT CONFIGURATION** **5**

**8 CORE — INTENT ARCHITECTURE** **5**

**9 CORE — CHARACTER SIMULATION** **6**

**10 CORE — BALL SIMULATION** **6**

**11 CORE — TRAJECTORY** **7**

**12 CORE — DETECTION** **7**

**13 CORE — GAMEPLAY ACTION** **8**

**14 CORE — RALLY / MATCH STATE** **8**

**15 CORE — SYSTEM PROCESSING FLOW** **8**

**16 CORE — LOGGING** **9**

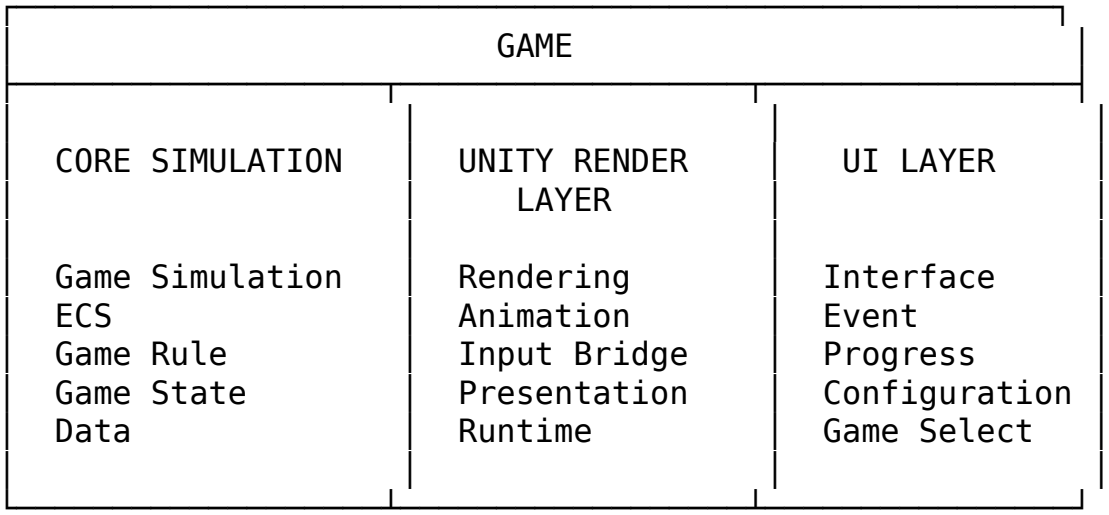
**17 CORE — SINGLE MATCH** **9**

**18 NHỮNG GÌ CORE CHƯA NÊN GHI LÀ ĐÃ HOÀN THÀNH** **9**

19 CORE — MULTI-MATCH 10
20 CORE — MULTI-MATCH MEMORY MODEL 10
21 CORE — INPUT QUEUE 11
22 CORE — SERVER FRAME MODEL 11
23 CORE — SERVER EXECUTION 11
24 CORE — AI 11
25 CORE — PERFORMANCE / BENCHMARK 12
26 UNITY RENDER LAYER 12
27 UNITY — INPUT BRIDGE 13
28 UNITY — RENDER DATA 13
29 UNITY — RELEASE SCOPE 14
30 UI LAYER 14
31 UI — GAME ENTRY 14
32 UI — INFORMATION 15
33 UI — CONFIGURATION 15
34 UI — RELEASE SCOPE 15
35 API / DATA CONTRACT 15
36 FLOW TOÀN BỘ GAME 16
37 PHÂN CHIA CÔNG VIỆC GIỮA 3 TEAM 17
38 CORE — ĐÃ LÀM / TIẾP TỤC 17
39 UNITY — PHẠM VI TỐI THIỂU 18
40 UI — PHẠM VI TỐI THIỂU 18
41 RELEASE 1 — PHẠM VI 18
42 PHÁT TRIỂN SAU RELEASE 1 19
43 DEFINITION OF DONE 19
44 NGUYÊN TẮC SCOPE 20
45 TÓM TẮT KIẾN TRÚC 20

# 1. Tổng quan

Project được chia thành ba phần chính:




Ba phần được tách biệt để Core Simulation không phụ thuộc trực tiếp vào Unity Rendering hoặc UI.

Mục tiêu của kiến trúc là để Simulation có thể được sử dụng độc lập với lớp hiển thị, đồng thời có khả năng mở rộng từ một Match hiện tại sang nhiều Match và các môi trường Server/AI trong tương lai.

# 2. Ranh giới trách nhiệm

## Core Simulation

Core chịu trách nhiệm trả lời: **Game đang xảy ra chuyện gì?**

Core quản lý:

- Game State.
- Character State.
- Ball State.
- Gameplay Rule.
- Intent.
- Movement.
- Action.
- Trajectory.
- Rally.
- Match State.
- Simulation Tick.
- Simulation Data.

Core không chịu trách nhiệm:

- Rendering.
- Animation.

- UI.
- Presentation.

### Unity Render Layer

Unity chịu trách nhiệm trả lời: **Game đang được hiển thị như thế nào?**

Unity quản lý:

- Rendering.
- Animation.
- Character/Ball representation.
- Camera.
- Environment.
- Visual effects.
- Input bridge.
- Runtime integration.

Unity không quyết định Gameplay Rule hoặc Simulation State.

### UI Layer

UI chịu trách nhiệm trả lời: **Người dùng nhìn thấy và tương tác với giao diện như thế nào?**

UI quản lý:

- Interface.
- Event information.
- Progress information.
- Configuration.
- Selection.
- Game information.
- Các thao tác giao diện.

Khi người dùng chọn ván và nhấn Play:

$$\texttt{UI} \rightarrow \texttt{Selected Match / Configuration} \rightarrow \texttt{Unity Render Layer} \rightarrow \texttt{Core Simulation}$$

UI không trực tiếp thao tác ECS hoặc Gameplay State.

# 3. CORE SIMULATION

## 3.1. Mục tiêu

Core Simulation là phần quan trọng nhất của kiến trúc.

Mục tiêu là xây dựng một Simulation Engine có thể mô phỏng game độc lập với Rendering.

Kiến trúc hiện tại tập trung vào ECS/Data-Oriented Design, trong đó các loại dữ liệu runtime được tổ chức thành các pool để giảm việc truy cập dữ liệu phân tán và tạo nền tảng cho việc xử lý cache-friendly.

# 4. CORE — ECS FOUNDATION

**Trạng thái: ĐÃ CÓ**

Core đã có nền tảng ECS tự xây dựng bằng C++.

Các thành phần chính gồm:

- Entity.
- Component.
- ComponentPool.
- ComponentStorage.
- IntentStorage.
- Các Component Pool chuyên biệt cho gameplay.

Mục tiêu của cách tổ chức này là tập trung các Component cùng loại vào các vùng dữ liệu riêng, thay vì mỗi Entity chứa toàn bộ dữ liệu của nó.

Ví dụ:

```
CharacterPositionPool
CharacterSizePool
CharacterStatusPool
BallPositionPool
BallTrajectoryPool
...
```

Thay vì:

```
Entity
├── Position
├── Size
├── Status
├── ...
```

Thiết kế này tạo nền tảng cho Data-Oriented Processing.

# 5. CORE — DATA STORAGE

**Trạng thái: ĐÃ CÓ**

Core hiện đã có các nhóm dữ liệu phục vụ Simulation, bao gồm:

- Character Position.
- Character Size.
- Character Status.
- Jump Frame.
- Ball Position.

- Ball Trajectory.
- Ball Gameplay State.
- Rally State.
- Match Gameplay State.
- Character Intent.
- Các runtime state khác.

Các data này được quản lý thông qua các Storage/Pool thay vì phụ thuộc vào Unity Object.

# 6. CORE — CHUNK CONFIGURATION

**Trạng thái: ĐÃ CÓ**

Project đã có ChunkConfig để xác định cách tổ chức dữ liệu theo Match.

Các thông tin như số lượng Character trong một Match được quy định trong Configuration.

Mục đích:

$$\texttt{Match} \rightarrow \texttt{Chunk Configuration} \rightarrow \texttt{Data Layout} \rightarrow \texttt{Component Pool}$$

Điều này tạo nền tảng cho việc mở rộng từ một Match sang nhiều Match.

# 7. CORE — ENTITY / OBJECT CONFIGURATION

**Trạng thái: ĐÃ CÓ**

Entity trong Simulation được tổ chức theo index.

Các object gameplay chính gồm:

- Player.
- Teammates.
- Opponents.
- Net.
- Ball.

Các index này được dùng để truy cập dữ liệu trong Component Pool.

Đây là nền tảng cho cách truy cập dữ liệu O(1) theo entity/index.

# 8. CORE — INTENT ARCHITECTURE

**Trạng thái: ĐÃ CÓ**

Một trong những nguyên tắc chính của Simulation là tách Input/Decision khỏi Component State.

Flow:

```
Input → Intent → Component / Simulation State → Game Rule
```

Input chỉ là tầng đầu vào. Input không trực tiếp sửa Component. Intent là tầng trung gian để biểu diễn hành động cần thực hiện.

Ví dụ:

```
Input → Move / Jump / Hit → Intent → Simulation
```

AI trong tương lai cũng có thể tạo Intent theo cùng interface.

# 9. CORE — CHARACTER SIMULATION

**Trạng thái: ĐÃ CÓ**

Core hiện đã có các nhóm xử lý Character.

**Movement**

- Horizontal movement.
- Position update.
- Movement boundary.
- Movement trong trạng thái trên không.

**Jump**

- Jump frame.
- Remaining jump frame.
- Vertical position update.
- Gravity-like movement.

**Character State**

- Action state.
- Status.
- Remaining frame/time của state nếu cần.

**Action**

Các action gameplay đã được xử lý trong Core, bao gồm các nhóm như:

- Serve.
- Bump.
- Set.
- Spike.
- Jump.
- Movement.

# 10. CORE — BALL SIMULATION

**Trạng thái: ĐÃ CÓ**

Ball được mô phỏng bằng data trong Core.

Các thành phần chính:

- Ball Position.
- Ball Trajectory.
- Ball Gameplay State.
- Landing Position.
- Last Touch.
- Rally-related state.

Ball không phụ thuộc vào Unity để xác định trạng thái gameplay. Unity chỉ nhận kết quả Simulation để hiển thị.

# 11. CORE — TRAJECTORY

**Trạng thái: ĐÃ CÓ**

Core đã có hệ thống tính và cập nhật trajectory.

Các dạng trajectory hiện tại gồm:

- Linear.
- Parabolic.

Trajectory sử dụng các tham số như: $a$, $b$, $c$, speed, direction, landingX.

Trajectory được thay đổi dựa trên Action của Character.

Ví dụ:

$$\texttt{Bump} \rightarrow \texttt{Trajectory}$$

$$\texttt{Set} \rightarrow \texttt{Trajectory}$$

$$\texttt{Spike} \rightarrow \texttt{Trajectory}$$

Các Configuration liên quan đến distance/angle/trajectory được tách khỏi logic xử lý.

# 12. CORE — DETECTION

**Trạng thái: ĐÃ CÓ**

Core có cơ chế Detection giữa Character và Ball.

Các thông tin được sử dụng gồm:

- Position.
- Distance.
- Character Size.
- Angle.
- Distance Percentage.

Detection được sử dụng để xác định khả năng thực hiện các action liên quan đến Ball.

Detection thuộc Simulation chứ không phụ thuộc Unity Collision.

# 13. CORE — GAMEPLAY ACTION

**Trạng thái: ĐÃ CÓ**

Core xử lý các action gameplay từ Intent.

Ví dụ:

```
Intent → Bump → Update Ball / State
Intent → Set → Update Ball / State
Intent → Spike → Update Ball / Trajectory
```

Action được xử lý trong Simulation. Unity chỉ nhận kết quả để biểu diễn.

# 14. CORE — RALLY / MATCH STATE

**Trạng thái: ĐÃ CÓ NỀN TẢNG**

Core đã có các state phục vụ Rally và Match.

Bao gồm các nhóm thông tin:

- Rally State.
- Last Touch.
- Touch count.
- Ball Gameplay State.
- Match Gameplay State.
- Serve.
- Gameplay event.

Các luật gameplay tiếp tục được hoàn thiện trong quá trình phát triển.

# 15. CORE — SYSTEM PROCESSING FLOW

**Trạng thái: ĐÃ CÓ**

Core được tổ chức theo hướng xử lý theo từng phase/system.

Ở mức interface:

```
Input → Intent → Component → Gameplay Rule →
                    Simulation State
```

Các System xử lý theo từng nhóm dữ liệu thay vì một Entity phải xử lý toàn bộ logic của nó.

Ví dụ:

```
for all Input
    update Intent

for all Intent
    update Component
    update Game Rule
    update Simulation State
```

Mục tiêu là giữ processing theo data type và tận dụng Data-Oriented Storage.

# 16. CORE — LOGGING

**Trạng thái: ĐÃ CÓ**

Core đã có khả năng ghi dữ liệu gameplay ra CSV.

Các dữ liệu có thể phục vụ:

- Debug.
- Replay analysis.
- Gameplay analysis.
- Dataset generation.
- AI research.

Logging được tách khỏi gameplay presentation.

# 17. CORE — SINGLE MATCH

**Trạng thái: ĐÃ CÓ / ĐANG HOÀN THIỆN**

Core hiện có thể vận hành Simulation cho một Match.

Mô hình hiện tại có thể được khái quát:

$$\texttt{Match} \rightarrow \texttt{Input} \rightarrow \texttt{Intent} \rightarrow \texttt{Component} \rightarrow \texttt{Simulation Systems} \rightarrow \texttt{Game Rule} \rightarrow \texttt{Simulation State}$$

Đây là nền tảng Core hiện tại.

# 18. NHỮNG GÌ CORE CHƯA NÊN GHI LÀ ĐÃ HOÀN THÀNH

Các phần sau đã được xác định trong kiến trúc tương lai nhưng không nên ghi là đã hoàn thành trong implementation hiện tại:

- Multi-Match Runtime.
- Match Scheduler.
- Match × Frame Queue.
- Server Input Queue.
- Server Output Queue.

- N-2 Simulation.
- N-4 Output.
- Multi-client Server.
- Server Network Layer.
- Large-scale Simulation Benchmark.
- AI Runtime hoàn chỉnh.
- AI Training Pipeline hoàn chỉnh.

Đây là các bước phát triển tiếp theo.

# 19. CORE — MULTI-MATCH

**Trạng thái: PHÁT TRIỂN TƯƠNG LAI**

Mục tiêu:

`SimulationMatch(MatchIndex)` và `SimulationMatches(...)`

cho phép Simulation xử lý: Match 0, Match 1, Match 2, …, Match N.

Match Index kết hợp với ChunkConfig + ObjectConfig để xác định vị trí data tương ứng.

Mục tiêu là truy cập data của một Match theo index với chi phí cố định, thay vì phải tìm kiếm dữ liệu.

# 20. CORE — MULTI-MATCH MEMORY MODEL

**Trạng thái: PHÁT TRIỂN TƯƠNG LAI**

Kiến trúc dự kiến tổ chức dữ liệu theo Match × Frame.

Ví dụ:

| | **Frame** | | |
|---|---|---|---|
| | K-2 | K-1 | K |
| Match 0 | D | D | D |
| Match 1 | D | D | D |
| Match 2 | D | D | D |
| … | | | |

Chỉ giữ số lượng frame cần thiết.

Memory mục tiêu: $N \times K \times$ FrameData

Trong đó:

- $N$ = số Match.
- $K$ = số frame cần giữ.

# 21. CORE — INPUT QUEUE

**Trạng thái: PHÁT TRIỂN TƯƠNG LAI**

Input từ client được đưa vào Queue.

Ví dụ: Input 98, Input 99, Input 100.

Nếu chỉ cần giữ K frame gần nhất thì input cũ hơn sẽ bị loại.

Ví dụ $K = 3$, Current $= 100$:

- Keep: 98, 99, 100.
- Discard: 97, 96, …

Dữ liệu bị loại có thể được ghi vào CSV log.

# 22. CORE — SERVER FRAME MODEL

**Trạng thái: PHÁT TRIỂN TƯƠNG LAI**

Mô hình server dự kiến:

$$\texttt{Client} \rightarrow \texttt{Input N} \rightarrow \texttt{Queue} \rightarrow \texttt{Simulation N-2} \rightarrow \texttt{Output N-4} \rightarrow \texttt{Client}$$

Simulation và Output không cần chờ Input mới.

Nếu tại frame cần Simulation không có Input:

$$\texttt{No Input} \rightarrow \texttt{Continue Simulation}$$

Điều này giúp Simulation không bị block bởi Network Input.

# 23. CORE — SERVER EXECUTION

**Trạng thái: PHÁT TRIỂN TƯƠNG LAI**

Khi có nhiều client kết nối (Client, Client, Client, …, Client) có thể được phân nhóm theo Match và scheduling strategy.

Mỗi nhóm Match có thể được đưa vào execution unit phù hợp.

Mục tiêu:

$$\texttt{Receive Input} \rightarrow \texttt{Queue} \rightarrow \texttt{Simulation} \rightarrow \texttt{Output Queue} \rightarrow \texttt{Client}$$

Network Layer và Simulation Layer được tách biệt.

# 24. CORE — AI

**Trạng thái: ĐANG PHÁT TRIỂN / FUTURE**

Kiến trúc Intent cho phép AI trở thành một nguồn tạo Intent khác.

Mô hình:

$$\texttt{AI} \rightarrow \texttt{Intent} \rightarrow \texttt{Core Simulation}$$

AI không cần trực tiếp thao tác Component.

Hướng nghiên cứu dự kiến gồm:

- Thu thập dữ liệu người chơi.
- Phân tích hành vi.
- Feature extraction.
- Defensive behavior model.
- Attack decision model.
- AI runtime.
- Training.
- Evaluation.

AI là một hướng phát triển trên Simulation Engine, không phải điều kiện để hoàn thành Core gameplay cơ bản.

# 25. CORE — PERFORMANCE / BENCHMARK

**Trạng thái: PHÁT TRIỂN TƯƠNG LAI**

ECS/Data-Oriented architecture được lựa chọn với mục tiêu xử lý dữ liệu hiệu quả và cache-friendly.

Tuy nhiên cần phân biệt: **Thiết kế cache-friendly** $\neq$ **Đã benchmark**.

Do đó cần benchmark thực tế sau khi Multi-Match hoàn thiện.

Các mức có thể benchmark: 1 Match, 10 Matches, 100 Matches, 1000 Matches, …

Các chỉ số cần đo:

- Tick time.
- Throughput.
- Memory usage.
- Simulation capacity.
- Data access performance.

# 26. UNITY RENDER LAYER

## Phạm vi trách nhiệm

Unity Render Layer chịu trách nhiệm đưa Simulation vào môi trường Unity và biến Simulation Data thành hình ảnh/gameplay presentation.

Các công việc chính:

- Tích hợp Core Simulation với Unity.

- Giao tiếp với Core thông qua API/contract.
- Khởi tạo và điều khiển runtime Simulation theo contract.
- Gửi Input từ Unity về Core.
- Nhận Simulation Data từ Core.
- Chuyển Simulation Data thành các đối tượng hiển thị trong Unity.
- Hiển thị Character.
- Hiển thị Ball.
- Hiển thị Court và các gameplay object.
- Xử lý Animation dựa trên State/Action từ Core.
- Xử lý Camera.
- Xử lý Presentation và Visual Effect cần thiết.
- Đồng bộ Simulation Data với Render Loop của Unity.
- Xử lý Runtime/Integration Error.
- Đóng gói và Build game.

Unity không chịu trách nhiệm:

- Tính Gameplay Rule.
- Quyết định Ball State.
- Quyết định Character State.
- Tự mô phỏng lại Core.
- Trực tiếp thao tác ECS Storage.

# 27. UNITY — INPUT BRIDGE

Input của người chơi được Unity tiếp nhận và chuyển về Core.

Flow:

```
Player → Unity Input → Input Contract → Core → Intent
```

Unity không biến Input trực tiếp thành Gameplay Action.

Ví dụ:

```
Player presses Spike → Unity sends Input → Core
processes Intent → Core determines Spike → Unity
receives result → Unity plays animation
```

# 28. UNITY — RENDER DATA

Core là nguồn dữ liệu Simulation.

Unity nhận: Position, State, Action, Event, Frame, …

và chuyển thành: GameObject, Animator, Transform, Visual Effect, Camera / Presentation.

Unity không tạo một Gameplay State song song với Core.

# 29. UNITY — RELEASE SCOPE

Mức tối thiểu để Unity hoàn thành phần của mình:

```
Core Integration → Input → Receive Simulation Data →
Character / Ball Rendering → Animation → Camera →
Playable Game → Release Build
```

Các phần nâng cao như:

- Advanced VFX.
- Advanced Animation.
- Audio.
- Advanced Camera.
- Optimization.
- Editor Tools.
- Debug Visualization.

có thể do Unity team tự phân rã và phát triển thêm.

# 30. UI LAYER

## Phạm vi trách nhiệm

UI Layer chịu trách nhiệm xây dựng giao diện tương tác với người dùng và hiển thị thông tin của game.

UI không phải Gameplay Simulation Layer.

Các công việc chính:

- Xây dựng giao diện lựa chọn ván/trận.
- Hiển thị Configuration.
- Hiển thị Event.
- Hiển thị Progress.
- Hiển thị thông tin cần thiết cho người chơi.
- Hiển thị trạng thái của game.
- Hiển thị kết quả.
- Xử lý các nút và thao tác giao diện.
- Quản lý các lựa chọn của người dùng.
- Quản lý data configuration.
- Gửi configuration/control information theo contract.
- Chuyển người dùng từ UI sang gameplay khi chọn ván và Play.

# 31. UI — GAME ENTRY

Luồng chính:

```
Launch Game → UI → Chọn ván / Configuration → Play →
Unity Render Layer → Core Simulation
```

Sau khi người dùng nhấn Play, Unity Render Layer tiếp nhận và thực hiện phần gameplay integration/render.

UI không trực tiếp khởi tạo hoặc thao tác ECS.

# 32. UI — INFORMATION

UI có thể hiển thị:

- Event.
- Progress.
- Match information.
- Score.
- Status.
- Result.
- Các thông tin cần thiết khác.

Nguồn data được lấy thông qua contract đã thống nhất.

UI không tự suy luận lại Gameplay State nếu thông tin đó thuộc Core.

# 33. UI — CONFIGURATION

UI có thể cho phép người dùng lựa chọn các configuration cần thiết.

Flow:

```
User Selection → Configuration Data → Game Start →
                    Unity / Core
```

Configuration là data contract, không phải UI trực tiếp chỉnh Component.

# 34. UI — RELEASE SCOPE

Mức tối thiểu:

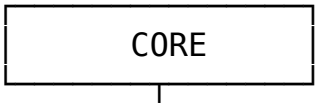
```
UI → Select Match → Select Configuration → Play →
Unity Gameplay → Display Event / Progress / Result
```

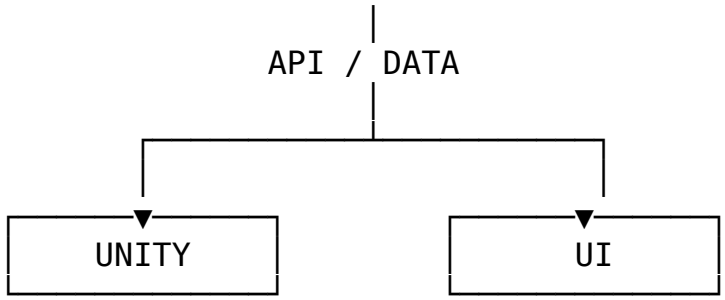
Các phần UX nâng cao do UI team tự phân rã và phát triển nếu cần.

# 35. API / DATA CONTRACT

Ba phần không truy cập trực tiếp implementation nội bộ của nhau.

Contract chính:





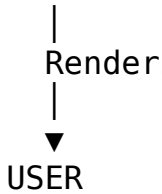

Core không expose: ComponentPool, ComponentStorage, Internal System, Internal ECS implementation thông qua interface gameplay bên ngoài.

Unity và UI chỉ nhận những dữ liệu cần thiết.

# 36. FLOW TOÀN BỘ GAME

Flow ở mức interface:

```
USER
  |
  ▼
 UI
  |
  Select Match
  |
  Configuration
  |
  Play
  |
  ▼
UNITY RENDER LAYER
  |
  Start / Connect Core
  |
  ▼
CORE SIMULATION
  |
  Input / Intent
  |
  Components
  |
  Gameplay Rule
  |
  Simulation State
  |
  ▼
Simulation Output
  |
  ▼
UNITY RENDER LAYER
```




UI trong quá trình đó có thể tiếp tục hiển thị các thông tin cần thiết như: Event, Progress, Score, Status, Result — nhưng không trở thành một phần của Simulation.

# 37. PHÂN CHIA CÔNG VIỆC GIỮA 3 TEAM

| Phần | Trách nhiệm chính |
|---|---|
| Core Simulation | Simulation, ECS, Data, Intent, Game Rule, Character, Ball, Match |
| Unity Render Layer | Integration, Rendering, Animation, Input Bridge, Camera, Presentation |
| UI Layer | Interface, Event, Progress, Configuration, Selection, Information |

# 38. CORE — ĐÃ LÀM / TIẾP TỤC

### Đã có

- ECS Foundation
- Component Storage
- Intent
- Character Simulation
- Ball Simulation
- Trajectory
- Detection
- Gameplay Action
- Rally / Match State
- Logging
- Single Match

### Cần tiếp tục

- Match Abstraction
- Multi-Match
- MatchIndex
- Data Layout hoàn chỉnh cho Multi-Match
- Frame Buffer
- Input Queue
- Output Queue
- Server Simulation
- Network Integration
- Performance Benchmark

**Hướng tương lai**

- AI Runtime
- AI Training
- Large-scale Simulation
- Server Scale

# 39. UNITY — PHẠM VI TỐI THIỂU

Unity cần hoàn thành:

- Core Integration
- Input Integration
- Simulation Data Integration
- Character Rendering
- Ball Rendering
- Animation
- Environment
- Camera
- Presentation
- Runtime Handling
- Release Build

Các phần nâng cao do Unity team tự quyết định.

# 40. UI — PHẠM VI TỐI THIỂU

UI cần hoàn thành:

- Interface
- Match Selection
- Configuration
- Event / Progress Display
- Information Display
- Play
- Result

Luồng quan trọng:

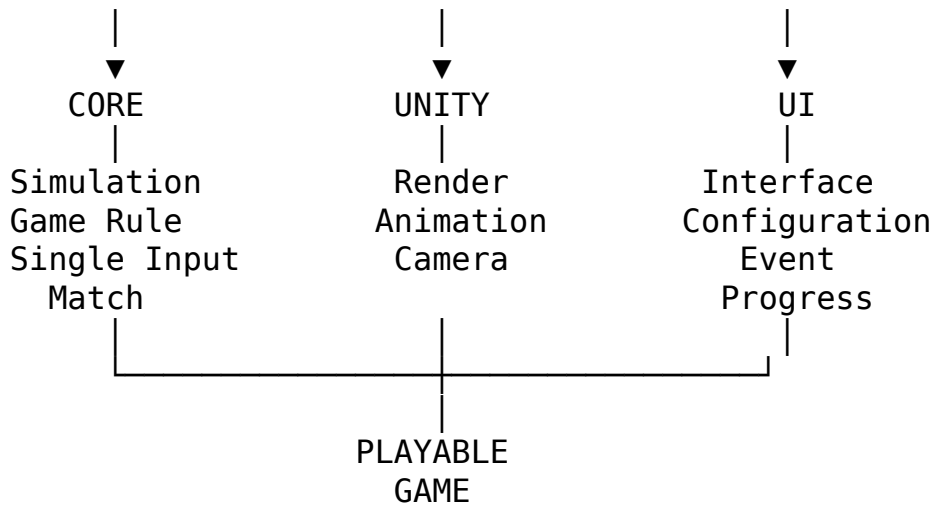
```
UI → Select Match → Play → Unity Render Layer
```

Các phần UX nâng cao do UI team tự quyết định.

# 41. RELEASE 1 — PHẠM VI

Mục tiêu Release 1 là tạo một game có thể chơi được.






Release 1 không bắt buộc phải có:

- Multi-Match Server.
- N-2/N-4 Server Model.
- Multi-client Server.
- Large-scale Simulation.
- Full AI Training.
- Reinforcement Learning.

# 42. PHÁT TRIỂN SAU RELEASE 1

Sau khi Playable Game hoàn thành:

```
Release 1 → Single Match Simulation → Multi-Match Simulation → Frame / Queue Architecture → Server Simulation → Multi-client → Large-scale Simulation → AI Training / Inference
```

Mỗi giai đoạn mở rộng Core mà không yêu cầu thay đổi bản chất của Unity Render Layer hoặc UI Layer nếu API/Data Contract vẫn giữ ổn định.

# 43. DEFINITION OF DONE

## Core

Một phiên bản Core được xem là đạt milestone khi:

- Simulation chạy ổn định.
- Gameplay Rule hoạt động.
- Input/Intent hoạt động.
- Character/Ball hoạt động.
- Output có thể được Unity sử dụng.
- Simulation không phụ thuộc Rendering.

### Unity

Một phiên bản Unity được xem là đạt milestone khi:

- Có thể tích hợp Core.
- Có thể gửi Input.
- Có thể nhận Simulation Output.
- Có thể render Character/Ball.
- Có Animation.
- Có gameplay presentation.
- Có thể build và chạy game.

### UI

Một phiên bản UI được xem là đạt milestone khi:

- Người dùng có thể lựa chọn ván/configuration.
- Có thể nhấn Play.
- Có thể chuyển sang Unity Gameplay.
- Có thể xem Event/Progress/Information.
- Có thể xem Result.

# 44. NGUYÊN TẮC SCOPE

Tài liệu này phân biệt rõ ba trạng thái:

**ĐÃ LÀM** — Đã có implementation hoặc có thể chứng minh bằng code hiện tại.

**PHÁT TRIỂN TIẾP** — Là những phần cần xây dựng tiếp để hoàn thiện kiến trúc/game.

**TƯƠNG LAI** — Là các hướng mở rộng như Server Scale và AI.

Không coi *Đã thiết kế* là *Đã implement*, và không coi *Có trong kiến trúc tương lai* là *Bắt buộc cho Release 1*.

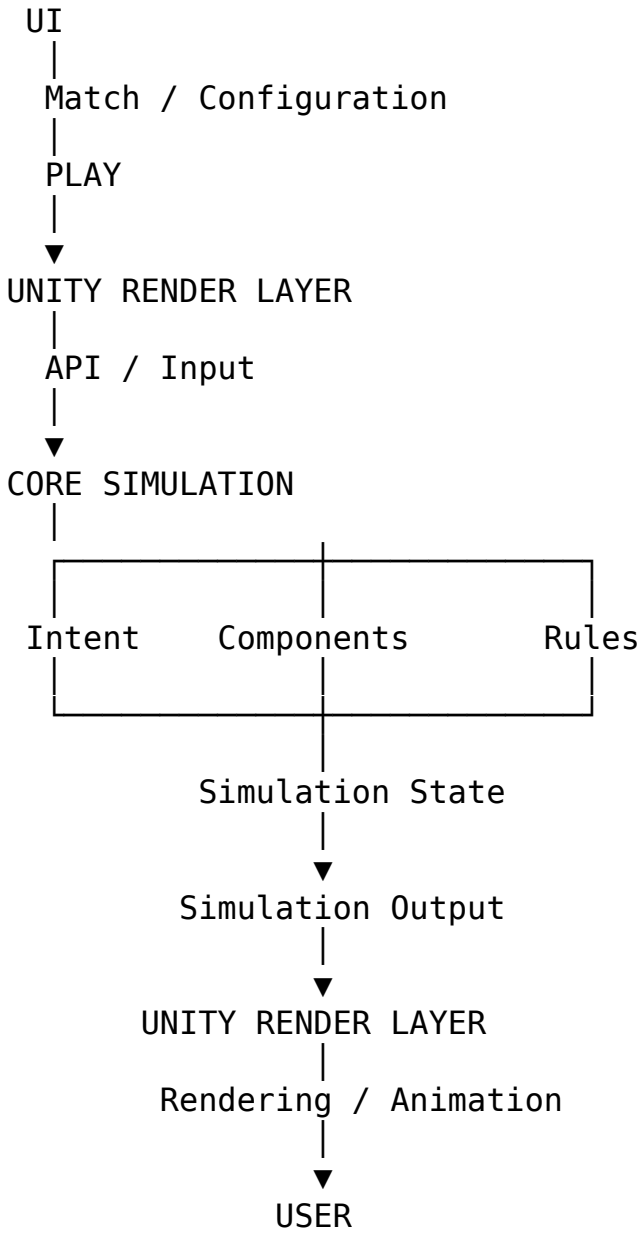
# 45. TÓM TẮT KIẾN TRÚC

Toàn bộ project có thể được hiểu đơn giản bằng ba câu:

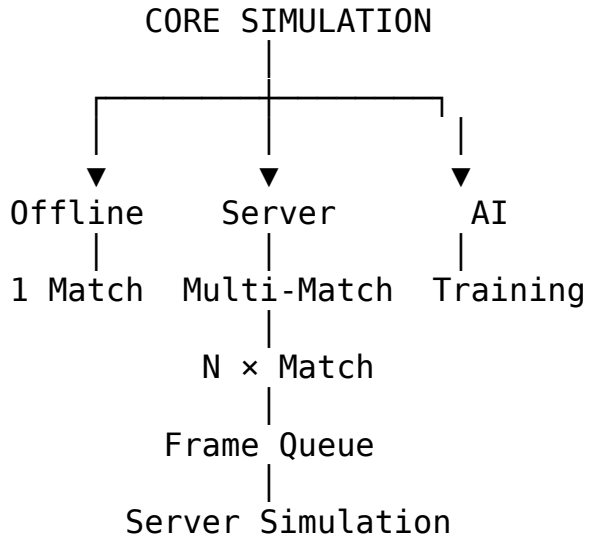
- Core Simulation quyết định game đang xảy ra chuyện gì.
- Unity Render Layer quyết định chuyện đó được hiển thị như thế nào.
- UI Layer cung cấp giao diện, thông tin và lựa chọn cho người dùng; khi người dùng chọn ván và Play, gameplay được chuyển sang Unity Render Layer.

**Kiến trúc cuối:**






**Về dài hạn:**




Đây là hướng phát triển từ một Core Simulation hiện tại thành một Simulation

Engine có thể phục vụ Offline, Server và AI trong tương lai.